

■ SalesRadar

Interview Questions & Answers

23 Questions | Hinglish + English | Full Coverage

Pehle Hinglish mein samjho, phir English mein batao

Section	Questions	Topics
Project Introduction	Q1–Q7	What/Why/How/Architecture/AI/ML
Technical Deep-Dive	Q8–Q14	REST/JPA/Design Patterns/DI/Cache/Error
Project Specific	Q15–Q19	CSV Upload/Data Isolation/PDF/KPI/Future
HR Questions	Q20–Q23	Learning/Failures/Timeline/Why Hire Me

SECTION 1: PROJECT INTRODUCTION

Q1. Tell me about your project / Apna project batao

■ HINGLISH — Pehle Samjho

SalesRadar ek AI-powered car sales analytics platform hai. Maine Java (Spring Boot) backend ke liye aur Python (Streamlit) frontend ke liye use kiya. 8 modules hain — CSV upload, KPI summary, yearly/monthly analytics, deep analytics, ML prediction, AI insights, aur PDF generation.

■ ENGLISH — Interviewer ko bolna

"SalesRadar is a full-stack automotive analytics platform bridging Java's enterprise capabilities with Python's data science ecosystem. It features 8 modules including AI-powered natural language querying via Spring AI + Ollama, ML-based forecasting using scikit-learn, and automated PDF reports via iTextPDF — all communicating through REST APIs."

Q2. What real-world problem does your project solve?

■ HINGLISH — Pehle Samjho

Car dealers ke 3 bade problems: (1) Manually reports banana padti hain — SalesRadar auto PDF generate karta hai. (2) Trends analyze karna mushkil — Deep analytics se brand/color/state patterns dikhte hain. (3) Future planning nahi hoti — ML prediction se next 1-5 saal forecast hota hai.

■ ENGLISH — Interviewer ko bolna

"Three key problems: First, manual reporting — managers waste hours on Excel reports; SalesRadar automates this with one-click PDF generation. Second, lack of trend visibility across dimensions like brand, color, state. Third, no forecasting capability causing inventory mismanagement — our ML module predicts future sales."

Q3. Why did you choose this tech stack?

■ HINGLISH — Pehle Samjho

Spring Boot — Java mein best REST framework hai. Spring AI — Java ke andar AI integrate ho gaya. Python/Streamlit — data science ke liye best. MySQL — structured data ke liye reliable. scikit-learn — simple ML ke liye best library.

■ ENGLISH — Interviewer ko bolna

"Deliberately chosen for polyglot architecture — each technology used where it excels. Spring Boot for enterprise REST APIs and DB management. Python for data science and visualizations. Spring AI chosen for clean LLM abstraction supporting local models via Ollama for data privacy. MySQL for reliable structured transactional data storage."

Q4. What was the most challenging part?

■ HINGLISH — Pehle Samjho

Sabse bada challenge data isolation tha. Do CSV upload kiye toh data mix ho gaya — AI ne 26 red cars bataye jab 13 hone chahiye the. Solution: DELETE endpoint jo upload se pehle DB clear karta hai. Doosra challenge iTextPDF import conflict — 130+ errors aaye wrong Document class ki wajah se.

■ ENGLISH — Interviewer ko bolna

"Most interesting challenge was data isolation — uploading two files merged results giving incorrect AI responses. Solved by implementing clear-before-upload mechanism via DELETE endpoint. Second challenge was iTextPDF class conflict — javax.swing.text.Document imported instead of com.itextpdf.text.Document causing 130+ compilation errors. Fixed by explicit correct imports."

Q5. How does the AI feature work?

■ HINGLISH — Pehle Samjho

User Streamlit pe question likhta hai. Ye Spring Boot ke /api/ai/ask endpoint pe jaata hai. Spring AI ye question Ollama ko bhejta hai — jo locally run hota hai. Ollama answer generate karta hai. Poora local — koi data bahar nahi jaata.

■ ENGLISH — Interviewer ko bolna

"Spring AI integrated with Ollama for local LLM inference. User question from Streamlit is sent via HTTP POST to Spring Boot AI controller. Spring AI forwards it to locally running Ollama instance which processes it through the LLM. Key advantage: complete privacy — no business data leaves the system, suitable for sensitive dealership data."

Q6. How does the ML prediction work?

■ HINGLISH — Pehle Samjho

Yearly sales data Spring Boot se fetch hota hai. scikit-learn ka Linear Regression model past data se pattern seekhta hai. Future years predict karta hai. R2 score model ki accuracy batata hai. Slider se 1-5 saal adjust kar sakte hain.

■ ENGLISH — Interviewer ko bolna

"scikit-learn LinearRegression trained on historical yearly sales. Years as feature (X), sales count as target (Y). Model learns linear trend and extrapolates for future years — configurable 1-5 years via slider. R2 score displayed as accuracy. For production, ARIMA or Prophet would provide better time-series accuracy with seasonality."

Q7. What is the architecture of your project?

■ HINGLISH — Pehle Samjho

2 main services: Spring Boot (Java — port 8080) aur Streamlit (Python — port 8501). Streamlit REST API calls karta hai Spring Boot ko. Spring Boot MySQL se data fetch karta hai aur Ollama se AI queries handle karta hai.

■ ENGLISH — Interviewer ko bolna

"Decoupled architecture with two independent services communicating via HTTP. Spring Boot on port 8080 follows Controller-Service-Repository layered pattern. Streamlit on 8501 calls APIs using requests library. MySQL for data persistence, Ollama for local AI, iTextPDF for reports as byte arrays. Separation allows each service to scale or modify independently."

SECTION 2: TECHNICAL DEEP-DIVE

Q8. What is REST API? How did you use it?

■ HINGLISH — Pehle Samjho

REST API ek communication style hai — HTTP methods use hote hain: GET (data fetch), POST (data bhejhe), DELETE (data delete). Hamare project mein 10 endpoints hain — yearly count, brand count, upload CSV etc.

■ ENGLISH — Interviewer ko bolna

"REST (Representational State Transfer) uses HTTP methods for web services. I implemented 10 REST endpoints — GET for analytics data, POST for CSV upload and AI queries, DELETE for data clearing. Each returns JSON wrapped in ApiResponse with success flag, message, and data."

Q9. What is JPA and how did you use it?

■ HINGLISH — Pehle Samjho

JPA Java objects ko database tables se map karta hai. CarSales entity class banai — @Entity, @Table, @Column annotations lagaye. Spring Data JPA automatic SQL generate karta hai. Custom queries ke liye @Query annotation use ki.

■ ENGLISH — Interviewer ko bolna

"JPA is the ORM specification for Java. I annotated CarSales class with @Entity, @Table, @Column to map it to car_sales table. JpaRepository provided built-in CRUD. Custom JPQL queries via @Query for complex aggregations like brand/state/color wise GROUP BY counts. Java records used for DTOs — YearlyCountDto and MonthlyCountDto."

Q10. What design patterns did you use?

■ HINGLISH — Pehle Samjho

1. Layered Architecture — Controller → Service → Repository. 2. DTO Pattern — YearlyCountDto, MonthlyCountDto. 3. Repository Pattern — CarSalesRepository extends JpaRepository. 4. Dependency Injection — Constructor injection everywhere. 5. Interface-Implementation — CarSalesService + CarSalesServiceImpl.

■ ENGLISH — Interviewer ko bolna

"Several patterns applied: Layered Architecture separates HTTP handling, business logic, and data access. DTO Pattern prevents exposing entities directly. Repository Pattern abstracts DB operations. Constructor Injection for loose coupling. Interface-Implementation follows Open/Closed SOLID principle."

Q11. What is Dependency Injection?

■ HINGLISH — Pehle Samjho

DI matlab class apni dependencies khud nahi banati — bahar se inject hoti hain. CarSalesController ko CarSalesService chahiye — Spring automatically inject kar deta hai constructor se. Loose coupling hoti hai — testing aasaan.

■ ENGLISH — Interviewer ko bolna

"DI is where objects receive dependencies from external sources rather than creating them. I used constructor injection throughout — Spring Boot's IoC container manages bean lifecycle. CarSalesController receives CarSalesService through constructor. Promotes loose coupling, testability, and Dependency Inversion SOLID principle."

Q12. What is caching? How did you implement it?

■ HINGLISH — Pehle Samjho

Caching — pehli baar API call hoti hai, result store ho jaata hai. Dobra same call pe stored result milta hai. Streamlit mein `@st.cache_data` use kiya. CSV upload pe `st.cache_data.clear()` se cache clear hota hai.

■ ENGLISH — Interviewer ko bolna

"Caching stores API responses in memory to avoid redundant network calls. `@st.cache_data` decorator on functions like `get_yearly_sales()` prevents repeated API calls. Cache explicitly cleared after CSV upload using `st.cache_data.clear()` ensuring fresh data. Significantly improves dashboard performance."

Q13. How did you handle errors?

■ HINGLISH — Pehle Samjho

Backend mein try-catch blocks, proper HTTP status codes. Frontend mein `ConnectionError` handle kiya. User ko clear messages — success/error/warning. Upload response mein `totalRecords`, `successCount`, `failedCount` return kiya.

■ ENGLISH — Interviewer ko bolna

"Multi-level error handling: Spring Boot uses try-catch during CSV parsing with appropriate HTTP status codes. Upload response includes `totalRecords`, `successCount`, `failedCount` for partial failure visibility. Streamlit catches `requests.exceptions.ConnectionError` gracefully showing user-friendly messages instead of raw stack traces."

Q14. What is @RestController vs @Controller?

■ HINGLISH — Pehle Samjho

@Controller traditional MVC mein view (HTML) return karta hai. @RestController = @Controller + @ResponseBody — directly JSON return karta hai. Hamare project mein REST APIs hain toh @RestController use kiya.

■ ENGLISH — Interviewer ko bolna

"@Controller returns view names in traditional Spring MVC. @RestController combines @Controller and @ResponseBody — automatically serializes return values to JSON. Since SalesRadar is a pure REST backend with no server-side views, @RestController was the appropriate choice for all three controllers."

SECTION 3: PROJECT-SPECIFIC QUESTIONS

Q15. How does CSV upload work end-to-end?

■ HINGLISH — Pehle Samjho

1. User CSV select karta hai. 2. DELETE /clear — purana data gone. 3. POST /upload-csv — multipart form data bheja. 4. Commons CSV parse karta hai. 5. Har row validate aur CarSales object banta hai. 6. MySQL mein saveAll(). 7. Response mein counts aate hain. 8. Streamlit cache clear hoti hai.

■ ENGLISH — Interviewer ko bolna

"CSV upload flow: DELETE clears existing data for isolation. POST sends file as multipart/form-data. Apache Commons CSV parses each row into field values. Each row maps to CarSales entity after validation. Valid records batch-saved via JPA saveAll(). Response returns totalRecords, successCount, failedCount. Streamlit cache cleared so analytics refresh with new data."

Q16. What is data isolation and why did you implement it?

■ HINGLISH — Pehle Samjho

Do CSV upload karne pe dono ka data mix ho gaya. AI ne 26 red cars bataye jab 13 hone chahiye the. Data isolation = naya data upload se pehle purana data delete karna. Isse har upload pe fresh, accurate analysis milti hai.

■ ENGLISH — Interviewer ko bolna

"Data isolation prevents contamination across uploads. Initially, uploading two files merged results — AI reported 26 red cars when current dataset had 13. Solved by clear-before-upload: frontend calls DELETE /api/car-sales/clear before each upload executing repository.deleteAll(). Critical for dealerships comparing different time periods or branch data."

Q17. How does the PDF generation work?

■ HINGLISH — Pehle Samjho

User "Generate PDF" click karta hai. Spring Boot mein PdfServiceImpl: iTextPDF Document create hota hai → data fetch hota hai → tables/paragraphs add hote hain → PDF bytes ByteArrayOutputStream mein store hote hain → response mein bytes bheje jaate hain. Streamlit download button dikhata hai.

■ ENGLISH — Interviewer ko bolna

"PdfServiceImpl creates iTextPDF Document with PdfWriter writing to ByteArrayOutputStream. Fetches yearly data, monthly data, KPI metrics from service layer. Structures into PdfPTables with styled headers. Document closed and byte array returned as ResponseEntity with Content-Type: application/pdf and Content-Disposition: attachment header. Streamlit presents native browser download button."

Q18. What is your KPI Summary page?

■ HINGLISH — Pehle Samjho

KPI = Key Performance Indicators. Ek page mein 9 important business metrics: Total Cars, Top Brand, Most Popular Color, Top State, Top Fuel, Top Payment, Avg Price, Top City. Ek API call se aata hai — `/api/car-sales/kpi-summary`. Managers ek glance mein sab dekhte hain.

■ ENGLISH — Interviewer ko bolna

"KPI Summary consolidates 9 critical business metrics into one view — solving the problem of navigating multiple reports. Backend aggregates all metrics in single service method: total count, top brand/color/state via GROUP BY queries, avg price via AVG query. Single API call minimizes network overhead. Mini donut charts show top 5 brand, color, state."

Q19. Future scope — what would you add?

■ HINGLISH — Pehle Samjho

1. ARIMA/Prophet — better time series prediction. 2. Docker — easy deployment. 3. JWT Authentication — user roles. 4. Anomaly detection — unusual sales alerts. 5. RAG — AI directly answers from uploaded CSV data.

■ ENGLISH — Interviewer ko bolna

"Planned enhancements: ARIMA or Prophet for time-series forecasting with seasonality. Docker containerization for deployment. JWT authentication with admin/viewer roles. Anomaly detection to auto-flag unusual patterns. RAG (Retrieval Augmented Generation) making AI answer directly from uploaded data for contextually accurate responses."

SECTION 4: HR & GENERAL QUESTIONS

Q20. What did you learn from this project?

■ HINGLISH — Pehle Samjho

1. Polyglot architecture — 2 languages ek saath. 2. Spring AI — Java mein AI integrate karna. 3. Data isolation ka importance — real bug tha. 4. PDF generation — programmatic documents banana. 5. Real ML deployment — notebook se bahar.

■ ENGLISH — Interviewer ko bolna

"Key learnings: Polyglot architecture — best solution combines languages playing to their strengths. Spring AI — LLM integration in enterprise Java apps. Data integrity importance — isolation bug taught to always consider data boundaries. Practical ML deployment beyond Jupyter notebooks. API design — consistent REST contracts between independent services."

Q21. How long did it take?

■ HINGLISH — Pehle Samjho

Lagbhag 3-4 hafte. Pehle hafte Spring Boot foundation. Doosre hafte Streamlit integration. Teesre-chautha hafte AI, ML, PDF features.

■ ENGLISH — Interviewer ko bolna

"Built over approximately 3-4 weeks with progressive iteration. Week 1: Spring Boot foundation and REST APIs. Week 2: Streamlit integration and charts. Weeks 3-4: Advanced features — Spring AI, ML prediction, PDF generation, KPI module, Deep Analytics. Iterative approach helped identify issues like data isolation early."

Q22. Did you face failures? How did you overcome?

■ HINGLISH — Pehle Samjho

Haan. 1. iTextPDF — 130+ errors aaye wrong import ki wajah se — systematic debugging se fix kiya. 2. Data isolation bug — AI galat answer de raha tha — root cause trace karke DELETE endpoint banaya. 3. ML accuracy kam thi — samjha ki synthetic data mein variation nahi thi, model theek hai.

■ ENGLISH — Interviewer ko bolna

"Yes, several failures. iTextPDF import conflict caused 130+ errors — traced to wrong Document class import, fixed by explicit iTextPDF imports. Data isolation bug gave incorrect AI responses — traced to missing data clearing, implemented DELETE endpoint solution. Low ML accuracy initially concerned me — analyzed and found it was expected with low-variance synthetic data, not a model defect. Systematic debugging over random trial-and-error was key."

Q23. Why should we hire you based on this project?

■ HINGLISH — Pehle Samjho

1. Full-stack — Java + Python dono aata hai. 2. Real problems solve kiye. 3. Modern tech — Spring AI, Ollama use kiya. 4. Architecture thinking — layered design.

■ ENGLISH — Interviewer ko bolna

"SalesRadar demonstrates: Full-stack versatility across Java backend and Python data science. Problem-solving mindset — identified real business problems and engineered solutions. Modern tech adoption — Spring AI and Ollama shows I stay current with emerging technologies. Architectural thinking — decoupled design with clear separation of concerns shows I think beyond making things work to making them maintainable and scalable."

QUICK REVISION — 2 MINUTE READ

Tech Stack (Interview se pehle yaad karo):

Backend	Spring Boot 3 + Spring AI + Ollama + MySQL + JPA + iTextPDF + Commons CSV
Frontend/ML	Python + Streamlit + Plotly + scikit-learn + Pandas + NumPy + Requests

8 Features (Ek ek yaad karo):

#	Feature	Key Point
1	CSV Upload	Data isolation — clear before upload
2	KPI Summary	9 metrics — ek API call mein
3	Yearly Analytics	Line + Bar charts — Plotly
4	Monthly Analytics	Year select karke monthly breakdown
5	Deep Analytics	Brand/Color/State — Pie + Bar charts
6	ML Prediction	Linear Regression — R2 score
7	AI Insights	Spring AI + Ollama — local LLM
8	PDF Report	iTextPDF — ByteArrayOutputStream

3 Key Problems Solved:

Problem	Solution
Manual reporting	Auto PDF generation via iTextPDF
Data mixing	DELETE endpoint before each upload
No forecasting	scikit-learn Linear Regression ML model

Architecture (1 line mein):

Streamlit (Python :8501) → REST APIs → Spring Boot (Java :8080) → MySQL + Ollama

Best of luck Alok! Tu ye project bahut achhe se explain karega! ■■